

Homework 1 (CST 3656)

Alex Gemann

1. I trust computers less after reading about floating point numbers because I was under the impression that something like entering a decimal number into the computer would be very simple and easy because computers are so powerful but I now know that it can be tricky for the computer in certain cases.

$$\begin{aligned} 2.(x-2)^9 &= x^9 + 9C_8 2^1 x^8 + 9C_7 2^2 x^7 + 9C_6 2^3 x^6 + 9C_5 2^4 x^5 + \\ &9C_4 2^5 x^4 + 9C_3 2^6 x^3 + 9C_2 2^7 x^2 + 9C_1 2^8 x + -2^9 = \\ &x^9 - 18x^8 + 144x^7 - 672x^6 + 2016x^5 - 4032x^4 + 5376x^3 - 4608x^2 \\ &+ 2304x - 512 \end{aligned}$$

In the plots, p_1 appears to exactly picture the function while the Horner's algorithm estimate of p_2 has a wider range of points because there's a significant amount of error.

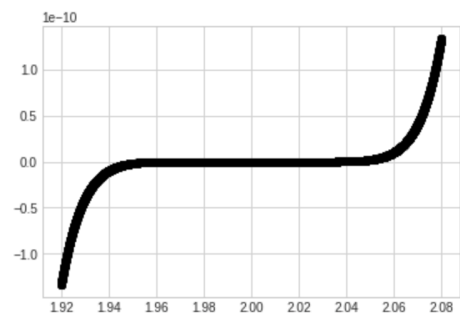
$$\begin{aligned}
 3. f_1(x) &= 1 - \cos(x) \\
 &= \frac{\sin^2(x)}{1 - \cos(x)} \cdot \frac{(1 + \cos(x))}{(1 + \cos(x))} \\
 &= \frac{1 - \cos^2(x)}{\sin^2(x)(1 + \cos(x))} \\
 &\Rightarrow = \frac{\sin^2(x)}{\sin^2(x)(1 + \cos(x))} \\
 &= \frac{1}{1 + \cos(x)} = f_2(x)
 \end{aligned}$$

4. I would amend the code that governs the robots to remove the 52 binary term limit on floating point numbers, meaning that whenever they need to convert a decimal that be converted into a finite number of binary terms they will get stuck in an infinite loop.

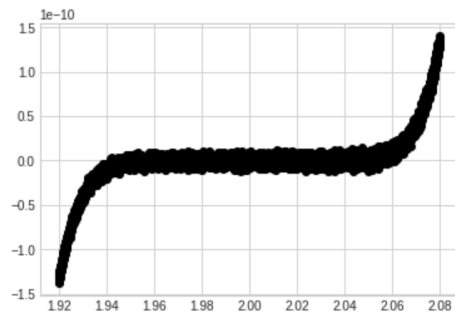
3 (continued). $f_1(x)$ loses its accuracy as x_k drops to 0 because when $k \geq 8$, the numerator of $f_1(x)$, $\sin^2(x)$, is so small that it rounds down to 0 which makes $f_1(x)$ 0 when really it's a tiny bit above 0. S. $f_2(x)$ reflects this because it doesn't have anything in it that's so small it would get rounded down to 0.

Python code for #2 and #3 on next page

```
[5]: %matplotlib inline
import matplotlib.pyplot as plt
plt.style.use('seaborn-whitegrid')
import numpy as np
import math
myList = []
num = 1.92001
while num < 2.08:
    myList.append(num)
    num += 0.00002
myVector = np.array(myList)
myNewVector = (myVector-2) ** 9
plt.plot(myVector, myNewVector, 'o', color='black');
```



```
[7]: %matplotlib inline
import matplotlib.pyplot as plt
plt.style.use('seaborn-whitegrid')
import numpy as np
import math
myList = []
num = 1.92001
while num<2.08:
    myList.append(num)
    num += 0.00002
myVector = np.array(myList)
myList2 = []
def hornersalgorithm(poly, n, x):
    result = poly[0]
    for i in range(1, n):
        result = result*x + poly[i]
    return result
polynomialList = [1,-18,144,-672,2016,-4032,5376,-4608,2304,-512]
polynomialArray = np.array(polynomialList)
for i in range(0,len(myVector)):
    myList2.append(hornersalgorithm(polynomialArray,len(polynomialArray),myVector[i]))
myVector2 = np.array(myList2)
plt.plot(myVector, myVector2, 'o', color='black');
```



```
[36]: import numpy as np
import math
myList=[]
for i in range(0,13):
    myList.append(10 ** -i)
myList2=[]
myList3=[]
for i in range(0,13):
    myList2.append((-math.cos(myList[i])+1)/((math.sin(myList[i])) ** 2))
    myList3.append(1/(1+math.cos(myList[i])))
for i in range(0,13):
    print("x: %.15f f1(x): %.15f f2(x): %.15f" %(myList[i],myList2[i],myList3[i]))

x: 1.000000000000000 f1(x): 0.649223205204762 f2(x): 0.649223205204762
x: 0.100000000000000 f1(x): 0.501252086288566 f2(x): 0.501252086288571
x: 0.010000000000000 f1(x): 0.500012500208481 f2(x): 0.500012500208336
x: 0.001000000000000 f1(x): 0.500000124992189 f2(x): 0.500000125000021
x: 0.000100000000000 f1(x): 0.499999998627931 f2(x): 0.500000001250000
x: 0.000010000000000 f1(x): 0.5000000041386852 f2(x): 0.500000000125000
x: 0.000001000000000 f1(x): 0.5000044450291337 f2(x): 0.500000000000125
x: 0.000000100000000 f1(x): 0.499600361081322 f2(x): 0.500000000000001
x: 0.000000010000000 f1(x): 0.000000000000000 f2(x): 0.500000000000000
x: 0.000000001000000 f1(x): 0.000000000000000 f2(x): 0.500000000000000
x: 0.000000000100000 f1(x): 0.000000000000000 f2(x): 0.500000000000000
x: 0.000000000010000 f1(x): 0.000000000000000 f2(x): 0.500000000000000
x: 0.000000000001000 f1(x): 0.000000000000000 f2(x): 0.500000000000000
x: 0.000000000000100 f1(x): 0.000000000000000 f2(x): 0.500000000000000
```